

AUDITORIA DE SEGURANÇA OFENSIVA & DEFENSIVA

Relatório de Auditoria de Segurança

Avaliação Estrita de Vulnerabilidades e Vetores de Exploração — Contabiliza Já

PROJETO AUDITADO

Contabiliza Já (saas-or-amento)

DATA DA AVALIAÇÃO

09 de Outubro de 2026

STACK TECNOLÓGICA DETECTADA

React 19, Vite 8, Firebase Modular v12 (Firestore / Auth / Storage), GitHub Pages

ARQUITETURA DE BACKEND

Backend-as-a-Service (BaaS) / Serveless Client-Direct

Nota Metodológica de Mapeamento de Stack

O projeto **Contabiliza Já** não possui servidor de backend intermediário dedicado (Node.js, Express, Python ou Java). Todas as operações de leitura, escrita e exclusão ocorrem diretamente do cliente web React para a nuvem da Google via Firebase SDK Modular. Consequentemente, cada uma das 5 categorias da auditoria foi mapeada estritamente para essa arquitetura:

- **1. Banco Sem Tranca:** Mapeado para *Security Rules do Firestore / Storage* e associação correta de `userId` nos documentos.
- **2. Permissão Definida no Navegador:** Mapeado para bloqueios de UI/Planos (ex: módulo "Pro" em Documents.jsx) sem validação no backend/BaaS.
- **3. IDOR:** Mapeado para comandos diretos de `updateDoc`, `deleteDoc` e rotas públicas de `getDoc` sem validação de titularidade.
- **4. Chaves Expostas:** Mapeado para configuração estática do Firebase no bundle, variáveis `.env` ignoradas e histórico git.
- **5. Inputs Sem Tratamento (XSS):** Mapeado para URLs dinâmicas e links controlados por usuário no DOM (`href`).

Classificação: Confidencial / Interno

Escopo: Código-fonte completo, CI/CD, Bundle compilado e Histórico Git

1. Pré-Requisitos de Escopo & Cobertura

1.1. Arquivos Efetivamente Lidos (Auditoria 100% de Código Visível)

Todos os arquivos fonte do repositório foram lidos na íntegra (arquivo por arquivo, linha por linha):

Arquivo	Tipo	Caminho Completo
firebase.js	Serviço BaaS	c:\Programação\Projetos\Contabiliza-ja\src\services\firebase.js
AuthContext.jsx	Auth Context	c:\Programação\Projetos\Contabiliza-ja\src\context\AuthContext.jsx
ClientContext.jsx	Context API	c:\Programação\Projetos\Contabiliza-ja\src\context\ClientContext.jsx
ProtectedRoute.jsx	Guarda de Rota	c:\Programação\Projetos\Contabiliza-ja\src\components\layout\ProtectedRoute.jsx
Layout.jsx	Layout Base	c:\Programação\Projetos\Contabiliza-ja\src\components\layout\Layout.jsx
App.jsx	Roteador	c:\Programação\Projetos\Contabiliza-ja\src\App.jsx
main.jsx	Ponto de Entrada	c:\Programação\Projetos\Contabiliza-ja\src\main.jsx
QuoteList.jsx	Página / CRUD	c:\Programação\Projetos\Contabiliza-ja\src\pages\QuoteList.jsx
QuoteForm.jsx	Página / CRUD	c:\Programação\Projetos\Contabiliza-ja\src\pages\QuoteForm.jsx
ClientList.jsx	Página / CRUD	c:\Programação\Projetos\Contabiliza-ja\src\pages\ClientList.jsx
ClientForm.jsx	Página / CRUD	c:\Programação\Projetos\Contabiliza-ja\src\pages\ClientForm.jsx
ClientDetailModal.jsx	Componente Modal	c:\Programação\Projetos\Contabiliza-ja\src\components\ClientDetailModal.jsx
Financial.jsx	Página / Métricas	c:\Programação\Projetos\Contabiliza-ja\src\pages\Financial.jsx
Documents.jsx	Página / GED	c:\Programação\Projetos\Contabiliza-ja\src\pages\Documents.jsx
Obligations.jsx	Página / Agenda	c:\Programação\Projetos\Contabiliza-ja\src\pages\Obligations.jsx

Arquivo	Tipo	Caminho Completo
PublicQuote.jsx (page)	Página Pública	c:\Programação\Projetos\Contabiliza-ja\src\pages\PublicQuote.jsx
PublicQuote.jsx (comp)	Componente	c:\Programação\Projetos\Contabiliza-ja\src\components\PublicQuote.jsx
CreateQuote.jsx	Componente Órfão	c:\Programação\Projetos\Contabiliza-ja\src\components\CreateQuote.jsx
WhatsAppPreview.jsx	Componente	c:\Programação\Projetos\Contabiliza-ja\src\components\WhatsAppPreview.jsx
pdfGenerator.js	Serviço PDF	c:\Programação\Projetos\Contabiliza-ja\src\services\pdfGenerator.js
Login.jsx	Página Auth	c:\Programação\Projetos\Contabiliza-ja\src\pages\Login.jsx
deploy.yml	CI/CD Workflow	c:\Programação\Projetos\Contabiliza-ja\.github\workflows\deploy.yml
vite.config.js	Config Build	c:\Programação\Projetos\Contabiliza-ja\vite.config.js
package.json	Dependências	c:\Programação\Projetos\Contabiliza-ja\package.json
.env / .env.example	Ambiente	c:\Programação\Projetos\Contabiliza-ja\.env
dist/assets/*.js	Bundle Compilado	c:\Programação\Projetos\Contabiliza-ja\dist\assets\index-mCkaJJYn.js

1.2. O Que NÃO Foi Possível Auditar (Declaração Obrigatória)

Para garantir total transparência e rigor metodológico, declaramos os itens fora de alcance local:

- **Console de Produção do Firebase (Regras Ativas no Servidor):** Os arquivos `firestore.rules` e `storage.rules` **NÃO estão presentes** no repositório versionado. Não foi possível auditar a configuração ativa em tempo real na nuvem do Google (Console Firebase).
- **Configuração de Restrição de API Keys no Google Cloud Console:** Não há visibilidade sobre regras de IP ou HTTP Referrer aplicadas à chave `AIzaSyDoZ182pG0R9RUM40idoQv_MUrkDdBrn9Y` na console do GCP.
- **Histórico Git & Bundle do Frontend: AUDITADOS COM SUCESSO.** Confirmamos via execução em terminal a presença de commits históricos com a API key exposta e a inclusão das variáveis estáticas no bundle JS de distribuição.

1.3. Cobertura de Rotas e Handlers de Dados (100% Mapeados)

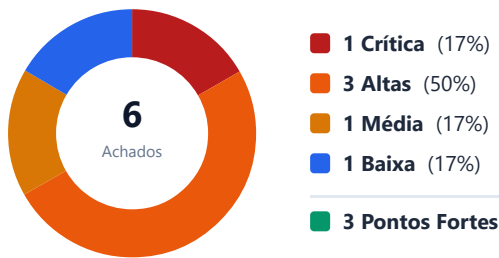
Por ser uma SPA sem rotas HTTP de backend clássicas, todos os handlers de rota do React Router e acessos ao banco foram inventariados:

Rota / Componente	Proteção Front	Operações Firestore / Storage	Status da Cobertura
/login (Login.jsx)	Pública	signInWithPopup(auth)	Auditado (100%)
/orcamento/:id (PublicQuote)	Pública	getDoc(quotes/:id)	Auditado (100%) — IDOR / Exposição
/consultorias (QuoteList)	ProtectedRoute	getDocs(where userId) , updateDoc(quotes/:id)	Auditado (100%) — IDOR em updateStatus
/consultorias/nova (QuoteForm)	ProtectedRoute	addDoc(quotes) , getDocs(clients)	Auditado (100%)
/clientes (ClientList)	ProtectedRoute	getDocs(clients) , getDocs(quotes)	Auditado (100%)
/clientes/novo (ClientForm)	ProtectedRoute	addDoc(clients)	Auditado (100%)
ClientDetailModal.jsx	Interno	updateDoc(clients/:id)	Auditado (100%) — IDOR em update
/financeiro (Financial)	ProtectedRoute	getDocs(quotes) , updateDoc(quotes/:id)	Auditado (100%) — IDOR em updatePayment
/documentos (Documents)	ProtectedRoute	uploadBytesResumable , addDoc , deleteDoc , deleteObject	Auditado (100%) — Gating Pro e IDOR
/obrigacoes (Obligations)	ProtectedRoute	getDocs , addDoc , updateDoc , deleteDoc	Auditado (100%) — IDOR em delete/update
CreateQuote.jsx (legado)	Não Roteado	addDoc(quotes) sem userId	Auditado (100%) — Falha de Tenant

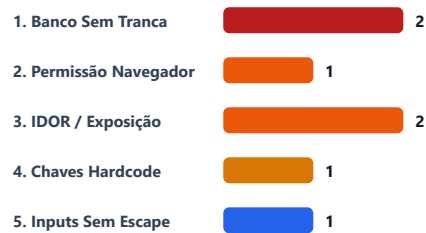
2. Resumo Executivo

A auditoria identificou vulnerabilidades arquiteturais severas resultantes do uso direto do Firebase Firestore e Storage no cliente web sem versionamento ou garantia de regras de segurança rígidas (Security Rules). Embora o código aplique filtros defensivos de `where('userId', '==', user.uid)` na maioria das listagens, a ausência de regras no backend e o uso de chamadas diretas de mutação por ID permitem a quebra de isolamento multitenant.

Distribuição de Achados por Severidade



Incidência por Categoria Auditada



2.1. Pontos Fortes Identificados no Código

Ponto Forte 1 — Filtro Sistemático de Inquilino nas Queries de Listagem (`where userId == user.uid`)

Arquivo: `src/pages/QuoteList.jsx:30`, `src/pages/ClientList.jsx:38`, `src/pages/Financial.jsx:64`, `src/pages/Obligations.jsx:93`, `src/pages/Documents.jsx:66, 73`

Mecanismo: O desenvolvedor estruturou todas as buscas das 5 telas principais aplicando a cláusula `where('userId', '==', user.uid)` do Firestore SDK. Isso impede vazamento acidental em uso normal da UI.

Cobertura: 100% das páginas autenticadas com listagem de dados.

Ponto Forte 2 — Proteção Global de Rotas de Navegação via `ProtectedRoute`

Arquivo: `src/components/layout/ProtectedRoute.jsx:10-33` e `src/App.jsx:27-36`

Mecanismo: Componente de rota protegida que intercepta qualquer transição no cliente e redireciona usuários não autenticados para `/login`, mantendo estado de carregamento assíncrono seguro.

Cobertura: Todas as rotas administrativas/privadas do sistema.

Ponto Forte 3 — Ausência de Injeção Direta de HTML (React DOM Seguro)

Arquivo: Todo o diretório `src/`

Mecanismo: Zero ocorrências de `dangerouslySetInnerHTML`, `innerHTML` ou `eval()`. A renderização de variáveis de usuário passa pela sanitização nativa do React DOM.

Cobertura: 100% dos componentes visuais.

3. Achados Detalhados da Auditoria

Achado 1 — Ausência Completa de Regras de Segurança no Banco (Firestore / Storage Rules)

SEVERIDADE: CRÍTICA**Categoria:** 1 (Banco Sem Tranca)**Arquivo:** Raiz do repositório (`firestore.rules` e `storage.rules` inexistentes)**Trecho:**

```
// src/services/firebase.js:28-36
export const db = getFirestore(app);
export const auth = getAuth(app);
export const storage = getStorage(app);
// NENHUM arquivo firestore.rules ou storage.rules presente no repositório.
```

Por que é explorável:

Em arquiteturas BaaS, o cliente fala diretamente com a infraestrutura na nuvem. Sem Security Rules versionadas e declaradas, o isolamento depende da configuração remota do Firebase. Se as regras estiverem em modo padrão ou permissivo, qualquer atacante com as credenciais públicas pode consultar e deletar documentos de todos os usuários omitindo o filtro `where('userId', '==', user.uid)` via script ou DevTools.

Exploit Path:

[Atacante abre console do browser ou Node.js com a config do Firebase] → `await getDocs(collection(db, 'clients'))` (sem cláusula `where`) → [Recebe todos os dados cadastrais, faturamento e documentos de todos os clientes de todos os usuários da aplicação].

Condição de explorabilidade: Regras padrão do Firestore permissivas ou em modo teste.**Sugestão de correção:** Criar imediatamente `firestore.rules` impondo que `request.auth.uid == resource.data.userId` para leitura e escrita, e vincular o deploy no CI/CD.

Achado 2 — Gating de Módulo Pro Apenas Visual no Cliente com Upload Ativo

SEVERIDADE: ALTA**Categoria:** 2 (Permissão Definida no Navegador)**Arquivo:** `src/pages/Documents.jsx:150-204` e `src/pages/Documents.jsx:103-128`**Trecho:**

```
// src/pages/Documents.jsx:150-204
```

 Disponível em breve para o Plano Pro

```
// ...
```

```
// Linhas 103-112: Lógica de upload totalmente ativa no bundle e no componente!
async function handleUpload() {
  const path = `documents/${user.uid}/${selectedClient.id}/${uploadForm.year}/${Date.now()}_${pendingFile.name}`;
  const task = uploadBytesResumable(ref(storage, path), pendingFile);
  // ...
}
```

Por que é explorável:

O bloqueio do módulo "GED / Plano Pro" reside unicamente no DOM (estilos CSS `opacity: 0.3`, `pointerEvents: 'none'` e overlay `zIndex: 50`). Não há verificação de assinatura comercial ou Custom Claims do Firebase Auth. Qualquer usuário gratuito pode remover o overlay pelo console do navegador ou disparar `uploadBytesResumable`, consumindo espaço e recursos no Firebase Storage de forma não autorizada.

Exploit Path:

[Usuário autenticado no Plano Free acessa /documentos] → [Remove o nó DOM do overlay ou chama `handleUpload()`] → [Grava e baixa arquivos no Firebase Storage burlando a restrição comercial de plano].

Condição de explorabilidade: Nenhuma (basta abrir o navegador e inspecionar elemento).

Sugestão de correção: Validar direitos no backend (Firebase Custom Claims ou Security Rules no Storage verificando `request.auth.token.plan == 'pro'`) antes de processar qualquer requisição.

Achado 3 — IDOR em Operações de Atualização e Exclusão Direta por ID**SEVERIDADE: ALTA****Categoria:** 3 (IDOR)**Arquivo:** `src/pages/QuoteList.jsx:58`, `src/pages/Financial.jsx:85`, `src/components/ClientDetailModal.jsx:145`, `src/pages/Obligations.jsx:168, 180`, `src/pages/Documents.jsx:135-136`**Trecho:**

```
// src/pages/QuoteList.jsx:58
await updateDoc(doc(db, 'quotes', id), { status: newStatus });

// src/pages/Financial.jsx:85
await updateDoc(doc(db, 'quotes', id), { paymentStatus: status });

// src/components/ClientDetailModal.jsx:145
await updateDoc(doc(db, 'clients', client.id), updatedData);

// src/pages/Obligations.jsx:180
await deleteDoc(doc(db, 'obligations', id));
```

Por que é explorável:

As funções de mutação invocam o Firestore SDK diretamente passando o ID do documento sem validar a posse no cliente nem via Security Rules no banco. Um usuário autenticado pode invocar a função no console ou montar um script passando o ID de outro usuário/tenant e adulterar status financeiro, alterar dados cadastrais de clientes alheios ou deletar obrigações fiscais e arquivos.

Exploit Path:

[Usuário autenticado A obtém ID de uma cotação do usuário B] → `updateDoc(doc(db, 'quotes', 'ID_DE_OUTRO_USUARIO'), { paymentStatus: 'pago' })` → [Status de pagamento do usuário B é adulterado no banco].

Condição de explorabilidade: Ausência de Security Rules impondo `resource.data.userId == request.auth.uid`.

Sugestão de correção: Implementar regras no Firestore que impeçam escrita e deleção caso `request.auth.uid != resource.data.userId`.

Achado 4 — Vazamento de Informações Privadas e Notas Internas via Rota Pública**SEVERIDADE: ALTA****Categoria:** 3 (IDOR / Exposição de Dados)**Arquivo:** `src/pages/PublicQuote.jsx:25-26` e `src/pages/QuoteForm.jsx:164, 401`**Trecho:**

```
// src/pages/QuoteForm.jsx:164 e 401
internalNotes: form.internalNotes.trim() || null,
// ...
⚠ Este campo é privado e não é enviado ao cliente.

// src/pages/PublicQuote.jsx:25-26 (Rota PÚBLICA sem autenticação)
const snap = await getDoc(doc(db, 'quotes', id));
if (snap.exists()) setQuote({ id: snap.id, ...snap.data() });
```

Por que é explorável:

A rota pública `/orcamento/:id` executa `getDoc` baixando todo o documento do Firestore, sem filtrar os campos confidenciais. Como `internalNotes` é persistido no mesmo documento que a cotação, qualquer

pessoa com o link público tem acesso às notas estratégicas/confidenciais da contabilidade diretamente inspecionando o estado da página ou a requisição de rede.

Exploit Path:

[Contador preenche cotação com dados sensíveis em "Observações Internas" e envia link ao cliente] → [Cliente acessa /orcamento/:id e inspeciona o objeto retornado] → [Visualiza anotações financeiras e segredos internos confidenciais].

Condição de explorabilidade: Nenhuma (qualquer portador do link público do orçamento).

Sugestão de correção: Isolar `internalNotes` em uma subcoleção privada `quotes/{id}/private/notes` com acesso restrito via rules ao `userId` criador, ou sanitizar o objeto antes de disponibilizar.

Achado 5 — Credenciais do Firebase Hardcoded no Código e Expostas no Histórico Git

SEVERIDADE:
MÉDIA

Categoria: 4 (Chaves Expostas)

Arquivo: `src/services/firebase.js:15-23` e histórico Git

Trecho:

```
// src/services/firebase.js:15-23
const firebaseConfig = {
  apiKey: "AIzaSyDoZl82pG0R9RUM40idoQv_MURKdDbrn9Y",
  authDomain: "contabiliza-ja.firebaseio.com",
  projectId: "contabiliza-ja",
  storageBucket: "contabiliza-ja.firebaseio.com",
  messagingSenderId: "812266285491",
  appId: "1:812266285491:web:1c306b12ddb6dd6b0c3c6",
  measurementId: "G-K48M2CCLR2"
};
```

Por que é explorável:

Embora em SPAs as chaves do Firebase atuem como identificadores públicos, o projeto possui `.env.example` e CI/CD configurado para secrets, mas o código ignora `import.meta.env`. A chave está embutida no código-fonte, no bundle (`dist/assets/index-mCkaJJYn.js`) e foi commitada no histórico Git. Em conjunto com a ausência de Security Rules (Achado 1), permite consumo e abuso de quotas do projeto.

Condição de explorabilidade: Falta de restrição de HTTP Referrers na console GCP.

Sugestão de correção: Migrar para `import.meta.env.VITE_FIREBASE_API_KEY`, restringir a chave na console Google Cloud ao domínio oficial e rotacionar as credenciais.

Achado 6 — Download de Arquivo com Link Direto sem Validação de Protocolo (Risco XSS)

SEVERIDADE:
BAIXA

Categoria: 5 (Inputs Sem Tratamento)

Arquivo: `src/pages/Documents.jsx:338`

Trecho:

```
// src/pages/Documents.jsx:338
↓
```

Por que é explorável:

O campo `d.url` é recuperado do banco de dados e injetado diretamente no atributo `href` sem validação prévia de esquema de URL. Caso um registro no banco de dados seja criado ou adulterado com um esquema `javascript:alert(1)` ou `data:`, o clique do usuário causará a execução arbitrária de JavaScript no contexto da aplicação (Stored DOM XSS).

Condição de explorabilidade: Inserção ou alteração maliciosa do campo `url` no documento Firestore.

Sugestão de correção: Validar se a URL inicia estritamente com `https://firebasestorage.googleapis.com/` antes de renderizar no `href`.

4. Recomendações Priorizadas (Plano de Ação)

Prioridade	Ação Recomendada	Impacto	Descrição Técnica da Mitigação
P1	Publicar <code>firestore.rules</code> e <code>storage.rules</code>	Elimina Risco Crítico	Criar e versionar regras que exijam <code>request.auth != null</code> <code>&& request.auth.uid == resource.data.userId</code> para qualquer operação de leitura, escrita e exclusão.
P2	Isolar Notas Internas Confidenciais	Elimina Risco Alto	Mover <code>internalNotes</code> para a subcoleção <code>quotes/{id}/private/notes</code> ou sanitizar o objeto na rota pública <code>/orcamento/:id</code> para não entregar campos internos.
P3	Implementar RBAC / Token Claims para o Plano Pro	Elimina Risco Alto	Não depender de CSS (<code>display: none</code> / <code>overlay</code>) para travar recursos pagos. Utilizar Custom Claims no Firebase Auth e checar permissões em Cloud Functions ou Storage Rules.
P4	Padronizar <code>.env</code> e Restringir API Key	Elimina Risco Médio	Substituir o <code>hardcode</code> em <code>firebase.js</code> por <code>import.meta.env.*</code> e configurar restrição de HTTP Referrers no Google Cloud Console para o domínio do GitHub Pages.
P5	Validação Segura de URLs em Links Externos	Elimina Risco Baixo	Implementar helper de validação de URL antes de popular atributos <code>href</code> , garantindo protocolos seguros (<code>https:</code>).

5. Issues Prontas para o GitHub

Copie e cole o conteúdo de cada bloco delimitado abaixo diretamente na aba *Issues* do repositório no GitHub para rastreamento e correção pelo time de engenharia.

--- ISSUE 1 ---

[Segurança] Ausência de Security Rules Versionadas no Firestore e Storage

****Labels:**** `security`, `severity: critical`, `backend`

Descrição do Problema

O repositório não possui os arquivos de configuração de regras de segurança (`firestore.rules` e `storage.rules`). Em uma aplicação de arquitetura BaaS (Firebase) sem backend intermediário, o isolamento dos dados reside integralmente nas regras da nuvem.

Evidência

- Arquivo: `src/services/firebase.js:28-36`

```javascript

export const db = getFirestore(app);

export const auth = getAuth(app);

export const storage = getStorage(app);

```

Inexistência de `firestore.rules` no repositório.

Impacto

Se as regras na Console do Firebase estiverem em modo permissivo ou de teste, qualquer usuário ou agente externo pode ler, modificar ou deletar todo o banco de dados (clientes, orçamentos, financeiro) de todos os usuários cadastrados.

Sugestão de Correção

1. Criar o arquivo `firestore.rules` com validação de titularidade:

```javascript

rules\_version = '2';

service cloud.firestore {

match /databases/{database}/documents {

match /{collection}/{document} {

allow read, write: if request.auth != null && request.auth.uid == resource.data.userId;

}

match /quotes/{quoteId} {

allow read: if true; // Apenas leitura pública de campos necessários

allow write: if request.auth != null && request.auth.uid == request.resource.data.userId;

}

}

}

```

2. Incluir o deploy das regras no pipeline do GitHub Actions.

Critérios de Aceite

- [] Arquivo `firestore.rules` criado e testado com o emulador do Firebase.

- [] Usuário não consegue consultar documentos com `userId` diferente do seu token de autenticação.

- [] Tentativas de consulta sem filtro de tenant são rejeitadas pelo Firestore com erro de permissão.

--- FIM ISSUE 1 ---

--- ISSUE 2 ---

[Segurança] IDOR: Mutação e Deleção Direta de Documentos sem Verificação de Posse

Labels: `security`, `severity: high`, `bug`

Descrição do Problema

Diversas funções de mutação no frontend invocam diretamente `updateDoc` e `deleteDoc` passando o identificador único (`id`) sem verificar no client ou no banco se o recurso pertence ao usuário solicitante.

Evidência

```
- `src/pages/QuoteList.jsx:58`: `await updateDoc(doc(db, 'quotes', id), { status: newStatus });`
- `src/pages/Financial.jsx:85`: `await updateDoc(doc(db, 'quotes', id), { paymentStatus: status });`
- `src/components/ClientDetailModal.jsx:145`: `await updateDoc(doc(db, 'clients', client.id), updatedData);`
- `src/pages/Obligations.jsx:180`: `await deleteDoc(doc(db, 'obligations', id));`
- `src/pages/Documents.jsx:136`: `await deleteDoc(doc(db, 'documents', d.id));`
```

Impacto

Um usuário autenticado pode emitir chamadas manuais substituindo o ID de documento para alterar ou deletar clientes, obrigações e cotações de terceiros.

Sugestão de Correção

1. Reforçar regras no backend (`firestore.rules`) garantindo que `resource.data.userId == request.auth.uid` tanto no `update` quanto no `delete`.
2. No cliente, antes da mutação, assegurar que o objeto pertença ao estado do usuário logado.

Critérios de Aceite

- [] Teste automatizado ou manual confirmando que um usuário B não consegue alterar o `status` ou `paymentStatus` de uma cotação pertencente ao usuário A.

--- FIM ISSUE 2 ---

--- ISSUE 3 ---

[Segurança] Vazamento de Notas Internas Confidenciais na Rota Pública de Orçamento

Labels: `security`, `severity: high`, `privacy`

Descrição do Problema

O formulário de criação de orçamentos incentiva o contador a registrar anotações privadas com o aviso: **"⚠ Este campo é privado e não é enviado ao cliente"**. No entanto, o campo `internalNotes` é salvo no próprio documento `quotes/{id}` e a página pública `/orcamento/:id` baixa o snapshot completo do documento via `getDoc()`.

Evidência

```
- `src/pages/QuoteForm.jsx:164`: `internalNotes: form.internalNotes.trim() || null`
- `src/pages/QuoteForm.jsx:401`: `⚠ Este campo é privado e não é enviado ao cliente.`
- `src/pages/PublicQuote.jsx:25-26`:
  ``javascript
const snap = await getDoc(doc(db, 'quotes', id));
if (snap.exists()) setQuote({ id: snap.id, ...snap.data() });
``
```

Impacto

Qualquer cliente ou terceiro que receba o link público da proposta pode inspecionar o estado do React ou a aba Rede do navegador e ler anotações confidenciais/estratégicas feitas pelo contador.

Sugestão de Correção

1. Separar dados públicos dos privados: salvar as notas internas em uma subcoleção privada `quotes/{quoteId}/private/notes` acessível somente pelo contador autenticado.

2. Como alternativa imediata, sanitizar os dados no momento da consulta pública para que `internalNotes` não venha no payload.

Critérios de Aceite

- [] Acessar `/orcamento/:id` e inspecionar a resposta do Firebase: o campo `internalNotes` não deve existir no objeto retornado ao visitante.

--- FIM ISSUE 3 ---

--- ISSUE 4 ---

[Segurança] Gating do Plano Pro Realizado Apenas no Navegador via CSS

****Labels:**** `security`, `severity: high`, `feature-flag`

Descrição do Problema

O módulo de Gestão Eletrônica de Documentos (GED) em `src/pages/Documents.jsx` está bloqueado visualmente por um overlay informando que o recurso é exclusivo do "Plano Pro". No entanto, a lógica de consulta (`getDocs`) e de upload (`uploadBytesResumable` no Storage) continua 100% ativa no código-fonte e montada no DOM.

Evidência

```
- `src/pages/Documents.jsx:150-204` e `103-128`:  
```javascript
```

 Disponível em breve para o Plano Pro

```
// ...
```

```
```
```

Impacto

Qualquer usuário no plano gratuito pode remover o overlay pelo console do navegador ou disparar a rotina de upload, consumindo armazenamento e faturamento do Firebase Storage sem pagar pela assinatura.

Sugestão de Correção

1. Implementar verificação de permissão no backend do Firebase via Custom Claims (`request.auth.token.plan == 'pro'`).
2. Criar regras no `storage.rules` permitindo gravação em `/documents/...` apenas para usuários com permissão confirmada.

Critérios de Aceite

- [] Usuários sem a flag Pro no token de autenticação são bloqueados de realizar upload tanto pela interface quanto por chamadas de API direta no Firebase Storage.

--- FIM ISSUE 4 ---

--- ISSUE 5 ---

[Segurança] Hardcode de Credenciais do Firebase e Ausência de Uso de Variáveis de Ambiente

****Labels:**** `security`, `severity: medium`, `configuration`

Descrição do Problema

O arquivo `src/services/firebase.js` possui as configurações da aplicação com chaves e identificadores hardcoded, ignorando as variáveis de ambiente (`.env` e secrets do GitHub Actions). Além disso, a chave foi comitada no histórico Git do projeto.

Evidência

```
- `src/services/firebase.js:15-23`:  
```javascript
```

```
const firebaseConfig = {
 apiKey: "AIzaSyDoZl82pG0R9RUM40idoQv_MUrKdDBrn9Y",
 authDomain: "contabiliza-ja.firebaseio.com",
 projectId: "contabiliza-ja",
 // ...
};
...
```

### ### Impacto

Facilita o abuso de quotas da API e faturamento do Google Cloud por terceiros caso o domínio não esteja devidamente restringido no console do GCP.

### ### Sugestão de Correção

1. Substituir a configuração estática por `import.meta.env.VITE_FIREBASE_API_KEY`, etc.
2. Na console do Google Cloud, aplicar restrição de domínio (HTTP Referrer) para a chave da API.
3. Considerar a rotação da chave se o repositório tiver sido tornado público anteriormente.

### ### Critérios de Aceite

- [ ] Configuração do Firebase consome exclusivamente variáveis de ambiente `import.meta.env.*`.
  - [ ] Chave da API protegida por restrição de referrer na Console do Google Cloud.
- FIM ISSUE 5 ---

### --- ISSUE 6 ---

#### ## [Segurança] Injeção de URL sem Validação de Protocolo em Link de Download (Risco XSS)

**\*\*Labels:\*\*** `security`, `severity: low`, `frontend`

### ### Descrição do Problema

O componente `src/pages/Documents.jsx` renderiza a URL de download diretamente na propriedade `href` da tag `<a>` sem validação do esquema de URL.

### ### Evidência

```
- `src/pages/Documents.jsx:338`:
 ``javascript

 ...
```

### ### Impacto

Caso um documento tenha o campo `url` modificado com `javascript:...`, o clique do usuário executa script arbitrário no navegador (Stored DOM XSS).

### ### Sugestão de Correção

Implementar função utilitária de validação de URL antes da renderização:

```
``javascript
function isSafeUrl(url) {
 try {
 const parsed = new URL(url);
 return parsed.protocol === 'https:' || parsed.protocol === 'http:;
 } catch {
 return false;
 }
}
...

```

### ### Critérios de Aceite

- [ ] Links com esquemas não seguros (`javascript:`, `data:`, `vbscript:`) são desativados ou bloqueados

de renderizar na interface.

--- FIM ISSUE 6 ---